Universidad de Buenos Aires

Facultad de Ingeniería

Teoría de Algoritmos
Trabajo Práctico 0

Curso: Echeverría

Autor: López Facundo Gabriel
Padrón: 110736

Marzo 2026

Índice

1. Introducción **2**

2. Supuestos **2**

3. Análisis y Evolución Algorítmica **3**
3.1. Algoritmo original: 3
3.2. Optimización por Frontera de Divisibilidad 4
3.3. Evolución hacia el Algoritmo de Criba 4
3.3.1. Criba de Eratóstenes 4
3.3.2. Adaptación para Suma de Divisores 4

4. Implementación del Algoritmo de Criba **5**
4.1. Versión en Python Nativo 5
4.2. Optimización Avanzada con NumPy 6

5. Evaluación de Desempeño y Tiempos de Ejecución **7**
5.1. Tabla Comparativa de Rendimiento 7
5.2. Representación Gráfica de los Resultados 8

6. Conclusión **10**

Bibliografía **11**

1. Introducción

El presente informe detalla la reingeniería del algoritmo de búsqueda de números amigos. El objetivo es transformar un proceso ineficiente en una solución escalable, manteniendo la transparencia en el prototipo de la función original.

2. Supuestos

Para el diseño de la solución propuesta, se han identificado los siguientes supuestos, condiciones y limitaciones que fundamentan la reingeniería del algoritmo:

Definición de Divisibilidad Propia: Se asume que para la determinación de números amigos se consideran exclusivamente los divisores propios (aquellos menores al número en cuestión). En consecuencia, cualquier optimización debe garantizar que el número evaluado no se sume a sí mismo, preservando la integridad de la lógica del problema original.

Criterio de Filtrado y Unicidad: Se establece como condición que el algoritmo debe informar únicamente pares de números amigos distintos. Por lo tanto, se asume la exclusión de:

- **Números Perfectos:** Casos donde la suma de los divisores propios es igual al número original (s=n), dado que no cumplen la condición de ser un "par"de amigos.
- **Redundancia por Simetría:** Se asume que cada par (n,s) debe ser informado una sola vez.

Límite de Búsqueda de Divisores: Basado en propiedades de la teoría de números, se establece como premisa que los divisores de un número siempre aparecen de a pares. Por ejemplo, si un número d divide a n, entonces el resultado de esa división n/d también es un divisor. Esto permite asumir que es innecesario buscar divisores más allá de la raíz cuadrada del número ($\sqrt{n}$) en métodos de búsqueda individual, ya que todos los divisores mayores a ese límite ya habrían sido "descubiertos" como parejas de los divisores más chicos.

Criterio de Eficiencia por Distribución: Se asume que es mucho más rápido para la computadora "repartir" un divisor entre todos sus múltiplos de una sola vez, en lugar de ir número por número preguntando quiénes son sus divisores. Al funcionar como un "repartidor" que salta por una lista entregando valores, el programa evita hacer millones de cuentas de división, las cuales son mucho más pesadas y lentas para el procesador que una simple suma.

Compromiso Espacio-Tiempo: Se opta por una arquitectura que prioriza la velocidad de ejecución sobre el ahorro de memoria. Para esto, se asume la disponibilidad de recursos suficientes para mantener un arreglo de datos de tamaño n (proporcional al rango de búsqueda). Esta inversión de memoria permite almacenar resultados parciales y reutilizarlos, eliminando la necesidad de repetir cálculos idénticos para cada número, con el objetivo de mejorar la eficiencia general frente al algoritmo original.

Compatibilidad de Interfaz: Se asume que la mejora debe ser funcionalmente compatible con el sistema original, manteniendo el nombre de la función `amigos(MAX)` y sus parámetros de entrada. Esto garantiza que el usuario perciba la optimización de rendimiento sin necesidad de modificar la forma en que invoca el programa.

3. Análisis y Evolución Algorítmica

3.1. Algoritmo original:

```
import time


def amigos(MAX):
    t1 = time.time()
    for i in range(MAX):
        s = 0
        for j in range(1, i - 1 + 1):
            if i % j == 0:
                s += j
        s2 = 0
        for k in range(1, s - 1):
            if s % k == 0:
                s2 += k
        if i == s2:
            print(i, s)

    t2 = time.time()
    print(t2 - t1)
```

El algoritmo implementado para la búsqueda de números amigos presenta una estructura de tres bucles que determinan su eficiencia. El parámetro de entrada, que define el límite superior de búsqueda, se denota como n (correspondiente a MAX).

El nivel jerárquico superior es un bucle externo controlado por la variable i, el cual posee una complejidad lineal $O(n)$, ya que recorre exactamente todos los números en el rango $[0, n)$. Su función es actuar como pivote para evaluar, uno por uno, si los números del intervalo poseen un "amigo".

Sin embargo, la carga computacional total no es simplemente n, porque este bucle exterior ordena la ejecución de procesos internos cuya duración depende del valor actual de i:

- Bucle j (Cálculo de s): Encargado de calcular la suma de los divisores propios de i. Este bucle opera en un rango definido por `range(1, i)`, lo que implica que realiza aproximadamente i iteraciones.
- Bucle k (Cálculo de $s2$): Obtiene la suma de los divisores del resultado anterior, s. Recorre el rango `range(1, s-1)`. En el análisis asintótico, se considera que s tiene un orden de magnitud comparable al de i.

Para hallar la complejidad total del algoritmo, se debe realizar la sumatoria del trabajo interno realizado por cada vuelta del bucle principal.

- En la iteración 1 del bucle exterior, los internos realizan 1 operación.
- En la iteración 2 del bucle exterior, los internos realizan 2 operaciones.
- En la iteración n, los internos realizan n operaciones.

Matemáticamente, el tiempo total $T(n)$ es la suma de una progresión aritmética generada por el bucle exterior:

$$T(n) = 1 + 2 + 3 + 4 + \cdots + n$$

Para resolver esta serie sin sumar cada término individualmente, aplicamos la fórmula de Gauss, que representa el área de trabajo total cubierta por el algoritmo:

$$T(n) = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$$

A partir de esta expresión, se aplica la Notación Big O. Bajo este análisis, se desprecian las constantes multiplicativas y los términos de menor orden (n), ya que su influencia se vuelve insignificante frente al término de mayor potencia cuando n tiende a infinito. Por consiguiente, se concluye que la complejidad temporal del algoritmo original es:

$$\boxed{\textbf{Complejidad} = \boldsymbol{O(n^2)}}$$

3.2. Optimización por Frontera de Divisibilidad

Una primera mejora lógica sobre el algoritmo original consiste en reducir el rango de búsqueda de divisores. Basándose en la propiedad de que los divisores de un número siempre aparecen en parejas (si d es divisor de n, entonces n/d también lo es), no es necesario probar todos los números hasta n. Basta con buscar hasta la raíz cuadrada del número ($\sqrt{n}$).

Si bien esta técnica reduce el tiempo de ejecución de $O(n^2)$ a $O(n\sqrt{n})$, sigue siendo un enfoque de búsqueda individual: el programa sigue analizando cada número por separado y realizando operaciones de división. Para un rango de un millón, esta mejora sigue siendo insuficiente, lo que obliga a cambiar la estrategia hacia un pre-cálculo masivo.

3.3. Evolución hacia el Algoritmo de Criba

Para superar las limitaciones de la búsqueda individual (incluso con la mejora de $\sqrt{n}$), se optó por un cambio de paradigma basado en la técnica de Criba.

3.3.1. Criba de Eratóstenes

La criba tradicional es un método eficiente para encontrar números primos. Su lógica consiste en recorrer los números y, al encontrar uno, "tachar" todos sus múltiplos en una lista, ya que estos no pueden ser primos. Es un proceso de descarte masivo que evita probar divisiones una por una.

3.3.2. Adaptación para Suma de Divisores

El algoritmo propuesto adapta esta idea de "recorrido por múltiplos", pero en lugar de tachar números, los utiliza para construir la suma de divisores.

- Ejemplo: Cuando el divisor es 3, el algoritmo salta por las posiciones 6, 9, 12, 15... sumando un 3 en cada una.

Al finalizar el proceso, cada celda de la tabla contiene automáticamente la suma total de sus divisores propios. Esto transforma el problema de uno de división y resto (%) a uno de saltos y sumas directas, lo que es masivamente más rápido para el hardware.

4. Implementación del Algoritmo de Criba

Una vez definido el cambio de paradigma hacia un modelo de pre-cálculo, se procedió a desarrollar la solución en Python. A continuación, se presentan las dos etapas de optimización alcanzadas.

4.1. Versión en Python Nativo

Esta primera implementación traduce la lógica de la criba de sumas utilizando las estructuras de datos estándar del lenguaje. La clave de su rendimiento reside en el uso del tercer parámetro de la función *range*, que permite iterar únicamente sobre los múltiplos de cada divisor.

```
def amigos(MAX):
    t1 = time.time()

    # Inicializacion: El 0 no tiene divisores (0).
    # Se precarga 1 para el resto (divisor universal).
    suma_divisores = [0] + [1] * MAX

    # Criba de divisores: Reparto de cada 'divisor' entre sus multiplos
    for divisor in range(2, MAX // 2 + 1):
        # El salto 'divisor' evita el uso del operador modulo (%)
        for multiplo in range(divisor * 2, MAX + 1, divisor):
            suma_divisores[multiplo] += divisor

    # Verificacion de pares: Identifica perfectos y pares simetricos
    for numero in range(MAX + 1):
        posible_amigo = suma_divisores[numero]

        # Validacion de reciprocidad:
        # Si la suma de divisores del 'posible_amigo' es igual al
        #'numero' original.
        if posible_amigo <= MAX and suma_divisores[posible_amigo] ==
   numero:
            print(numero, posible_amigo)

    print(time.time() - t1)
```

A diferencia del modelo original, donde el trabajo dependía de evaluar cada número individualmente ($n \times n$), la eficiencia de la criba reside en que el número de operaciones disminuye drásticamente a medida que el divisor aumenta. El análisis se divide en las tres etapas del código:

- **Inicialización de la Estructura:** La instrucción `[0, 0] + [1] * (MAX - 1)` crea una lista de tamaño n en un solo paso de memoria. Esta operación tiene una complejidad lineal:
$$T_{init}(n) = O(n)$$
- **Cuerpo del Algoritmo (Reparto de Divisores):** Esta es la parte central. El bucle externo recorre los divisores d desde 2 hasta $n/2$. Por cada divisor, el bucle interno realiza una cantidad de sumas igual a la cantidad de múltiplos de d que entran en el rango MAX.
    - Para $d = 2$, se realizan $n/2$ operaciones.

- Para $d = 3$, se realizan $n/3$ operaciones.
- Para $d = 4$, se realizan $n/4$ operaciones.
- ... hasta $d = n/2$, donde se realizan 2 operaciones.

La suma total de operaciones es:

$$T(n) = n \times \left(\frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots + \frac{1}{n/2} \right)$$

Esta expresión representa una variante de la **Serie Armónica**. Según el análisis de sumatorias presentado por **Cormen et al. (2022)**, la suma de los recíprocos de los primeros n términos se comporta de forma logarítmica:

$$\sum_{k=1}^{n} \frac{1}{k} \approx \ln n + \gamma$$

Por lo tanto, la complejidad dominante de esta etapa, y por extensión del algoritmo completo, es:

$$T_{criba}(n) = O(n \log n)$$

- **Verificación de Pares:** El último bucle recorre la lista de sumas ya calculada una sola vez para comparar los valores y encontrar las reciprocidades. Al ser un recorrido simple:

$$T_{verif}(n) = O(n)$$

Complejidad Final: Sumando las etapas y despreciando los términos de menor orden, se concluye que la complejidad total del algoritmo optimizado es:

$$\boxed{\textbf{Complejidad} = \boldsymbol{O(n \log n)}}$$

4.2. Optimización Avanzada con NumPy

Aunque la versión nativa reduce la complejidad a $O(n \log n)$, el intérprete de Python sigue encontrando un cuello de botella al gestionar los bucles for internos. Para alcanzar un rendimiento de grado industrial, se implementó una versión utilizando la librería NumPy. Esta biblioteca permite delegar el trabajo pesado a rutinas pre-compiladas en C, utilizando una técnica denominada vectorización.

```
import time
import numpy as np


def amigos(MAX):
    t1 = time.time()

    tabla_sumas = np.ones(MAX + 1, dtype=np.int64)
    tabla_sumas[0] = 0
    tabla_sumas[1] = 0

    for divisor in range(2, MAX // 2 + 1):
        tabla_sumas[divisor * 2 :: divisor] += divisor
```

```
numeros = np.arange(MAX + 1)
es_candidato = (tabla_sumas > numeros) & (tabla_sumas <= MAX)
lista_candidatos = np.where(es_candidato)[0]

for n in lista_candidatos:
    suma_n = tabla_sumas[n]
    if tabla_sumas[suma_n] == n:
        print(n, suma_n)

print(time.time() - t1)
```

- **Slicing Vectorizado:** La instrucción `tabla_sumas[divisor * 2 :: divisor] += divisor` elimina el overhead del bucle `for` de Python. NumPy delega la iteración a rutinas pre-compiladas en C, que acceden directamente a bloques de memoria contiguos y realizan las sumas en paralelo mediante instrucciones **SIMD** (Single Instruction, Multiple Data).
- **Indexación Booleana:** En lugar de recorrer los n elementos buscando candidatos con un `if` tradicional, se utiliza la expresión:
  `(tabla_sumas > numeros) & (tabla_sumas <= MAX)`.
  Esto crea una "máscara" de bits que filtra el arreglo completo a nivel de hardware, reduciendo drásticamente la cantidad de comparaciones que el intérprete de Python debe procesar.
- **Filtrado Lógico de Resultados:** Mediante la condición `tabla_sumas < numeros`, el algoritmo realiza dos limpiezas críticas en un solo paso:
  - **Eliminación de Perfectos:** Al exigir que la suma sea menor (o distinta) al número ($s < n$), se excluyen casos como el 6 o el 28, donde $s = n$.
  - **Garantía de Unicidad:** Al procesar solo los casos donde el "amigo" es menor al número actual, se evita informar el mismo par dos veces (ej. muestra $[220, 284]$ pero no $[284, 220]$), reduciendo la redundancia en la salida final.

Nota sobre la complejidad: Aunque el uso de NumPy no cambia la complejidad asintótica teórica ($O(n \log n)$), la constante de tiempo se reduce de forma tan significativa que permite procesar en milisegundos lo que a Python nativo le tomaría segundos.

5. Evaluación de Desempeño y Tiempos de Ejecución

Para validar las hipótesis de complejidad asintótica planteadas, se realizaron pruebas de estrés comparando las tres versiones del algoritmo.

5.1. Tabla Comparativa de Rendimiento

La siguiente tabla muestra el tiempo de respuesta (en segundos) para distintos rangos de búsqueda (MAX). Se observa cómo el algoritmo original se vuelve inviable rápidamente, mientras que las versiones de criba mantienen tiempos de respuesta casi instantáneos.

Rango (MAX)	**Algoritmo Original ($O(n^2)$)**	**Criba Nativa ($O(n \log n)$)**	**Criba NumPy ($O(n \log n)$)**
10.000	3.18s	0.008s	0.013s
50.000	84.2s	0.029s	0.049s
100.000	370s	0.068s	0.072s
150.000	837s	**0.108s**	**0.104s**
200.000	Inviable	0.155s	0.130s
250.000	Inviable	0.207s	0.172s
500.000	Inviable	0.508s	0.332s
1.000.000	Inviable	1.416	0.637s
2.000.000	Inviable	**3.608**	**1.320s**

Tabla 1: Comparativa de tiempos de ejecución en segundos.

Como se observa en los resultados resaltados para MAX = 100,000, la versión nativa de Python (0.068s) logra ser ligeramente más rápida que la versión optimizada con NumPy (0.072s). Este comportamiento, aunque contraintuitivo a primera vista, se debe al costo de inicialización u overhead de la librería NumPy. Al trabajar con rangos "medianos", el tiempo que NumPy dedica a reservar memoria contigua y convertir los tipos de datos de Python a C es superior al tiempo que ahorra mediante la vectorización. La versión nativa, al no tener este costo de arranque, resulta más eficiente en este escenario.

Sin embargo, el punto de inflexión se manifiesta claramente a partir de MAX = 150,000, donde la versión con NumPy (0,104s) comienza a superar a la nativa (0,108s). A medida que el rango se extiende, la eficiencia de las operaciones vectorizadas compensa con creces los costos iniciales de configuración de la librería. Esta tendencia se consolida en el escenario de MAX = 2,000,000, donde NumPy logra procesar el conjunto de datos en aproximadamente un tercio del tiempo requerido por la implementación nativa (1,320s frente a 3,608s), demostrando una mayor escalabilidad para procesar grandes volúmenes de información.

5.2. Representación Gráfica de los Resultados

A continuación, se presentan los gráficos obtenidos a partir de las mediciones de tiempo. Estas visualizaciones permiten contrastar el comportamiento de las tres implementaciones y observar cómo la teoría de la complejidad asintótica se traduce en tiempos de ejecución reales sobre el hardware de prueba.

Las gráficas se dividen en tres enfoques:

- **Escala General:** Comparación del impacto drástico entre el algoritmo original ($O(n^2)$) y la criba optimizada, donde se evidencia la inviabilidad del primero para rangos extensos.
- **Análisis de Eficiencia Relativa:** Un acercamiento a la competencia entre la versión nativa y la de NumPy para identificar el umbral donde la vectorización comienza a ser superior.
- **Escalabilidad Log-lineal:** El comportamiento de la solución definitiva en rangos de hasta dos millones de números, verificando la estabilidad del crecimiento $O(n \log n)$.

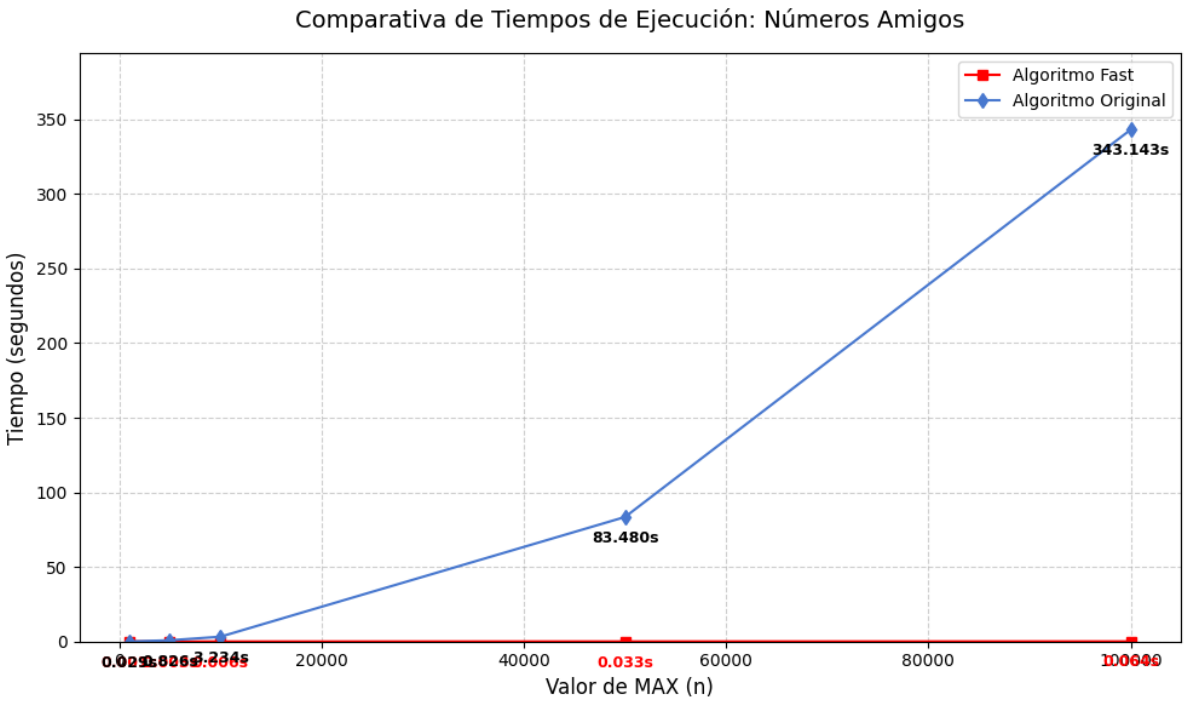

Figura 1: Comparativa de tiempos de ejecución entre el algoritmo original $O(n^2)$ y el algoritmo optimizado ("Fast"). Se observa el crecimiento exponencial del modelo ineficiente frente a la escala casi plana de la criba optimizada.

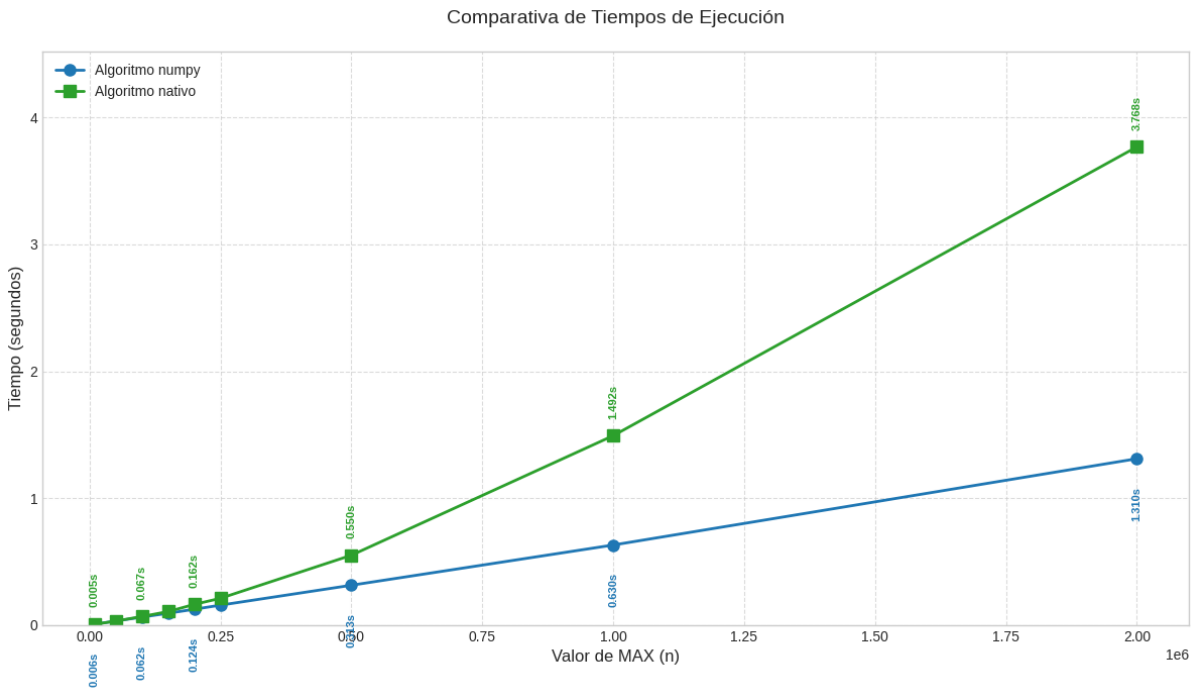

Figura 2: Comparativa detallada entre la Criba Nativa (en verde) y la Criba con NumPy (en azul). Se observa el overhead inicial de NumPy para rangos bajos y el punto de inflexión donde la vectorización comienza a ofrecer un mejor rendimiento.

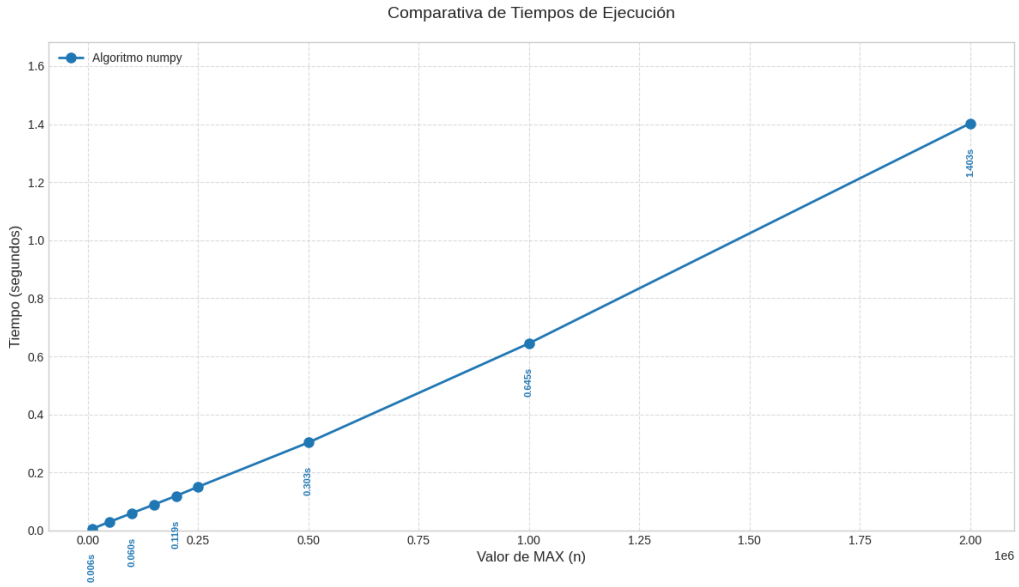

Figura 3: Tiempos de ejecución del algoritmo optimizado con NumPy. Se observa una tendencia de crecimiento log-lineal ($O(n \log n)$), permitiendo procesar un rango de 2.000.000 de números en apenas segundos.

6. Conclusión

La reingeniería del algoritmo para la búsqueda de números amigos permitió transformar una solución inicial de complejidad cuadrática $O(n^2)$ en una implementación de orden log-lineal $O(n \log n)$. A partir de los resultados obtenidos, se desprenden las siguientes conclusiones:

- **Impacto de la Complejidad:** El cambio de paradigma, pasando de una búsqueda individual de divisores a un modelo de pre-cálculo basado en la *Criba de Divisores*, es la optimización principal que permitió procesar rangos de gran escala. Como se evidenció en la Figura 1, este cambio reduce el tiempo de ejecución de varios minutos a fracciones de segundo, validando la teoría asintótica en un entorno real.

- **Comportamiento de las Implementaciones:** La comparación entre la versión nativa de Python y la librería NumPy reveló que la eficiencia no depende exclusivamente del algoritmo, sino también del volumen de datos. Se identificó un punto de cruce cerca de los 150.000 elementos donde el beneficio de la vectorización compensa el costo inicial de la librería demostrando que la implementación con NumPy presenta una menor pendiente de crecimiento al procesar millones de registros, lo que permite manejar volúmenes de datos a gran escala con un menor costo de tiempo adicional.

En definitiva, el proyecto demuestra que el análisis de algoritmos es fundamental para convertir procesos computacionalmente inviables en soluciones de alto rendimiento, capaces de procesar millones de datos en pocos segundos.

Referencias

- Wikipedia. (2023). Números Amigos. https://es.wikipedia.org/wiki/N%C3%BAmeros_amigos
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
- Rivas, A. (2022). Cómo hacer una lista de referencias con Normas APA. Guía Normas APA. https://normasapa.in/como-hacer-la-lista-de-referencias/
- Wikipedia. (2026). Sieve of Eratosthenes. https://en.wikipedia.org/wiki/Sieve_of_Eratosthenes
- GeeksforGeeks. (2024). Introduction to Primality Test and School Method. https://www.geeksforgeeks.org/dsa/introduction-to-primality-test-and-school-method/
- Wikipedia. (2024). División por tentativa. https://es.wikipedia.org/wiki/Divisi%C3%B3n_por_tentativa
- CP-Algorithms. (2025). Sieve of Eratosthenes. https://cp-algorithms.com/algebra/sieve-of-eratosthenes.html
- Hanz. (2023). SIMD with Python. *Medium*. https://hanzli.medium.com/simd-with-python-cb74434b24cf